

PROJECT TECHNICAL DOCUMENTATION

Autonomous 5-Channel IR Line Follower Robot

1. Abstract

This documentation details the hardware and software architecture of an autonomous line follower robot utilizing a 5-channel Infrared (IR) sensor array. The system uses a differential drive mechanism controlled by an L298N motor driver. The 5-IR configuration (2 Left, 1 Center, 2 Right) allows for high-resolution track reading, enabling smooth trajectory execution using a Proportional-Integral-Derivative (PID) algorithm or weighted error logic to eliminate sudden mechanical jerks.

2. Hardware Bill of Materials (BOM)

- Arduino Uno / Nano (Main Microcontroller)
- 5-Channel IR Sensor Array Module (TCRT5000 or similar)
- L298N Dual H-Bridge Motor Driver
- 2x DC Gear Motors (Differential Drive Setup)
- Robot Chassis with Castor Wheel
- External Power Source (7.4V - 11.1V LiPo Battery recommended)

3. Pin Mapping & Connections

A. 5-IR Sensor Array to Arduino

Sensor Position	IR Array Pin	Arduino Digital Pin
Far Left	IR_L2	D2
Near Left	IR_L1	D3
Center	IR_C	D4
Near Right	IR_R1	D11
Far Right	IR_R2	D12

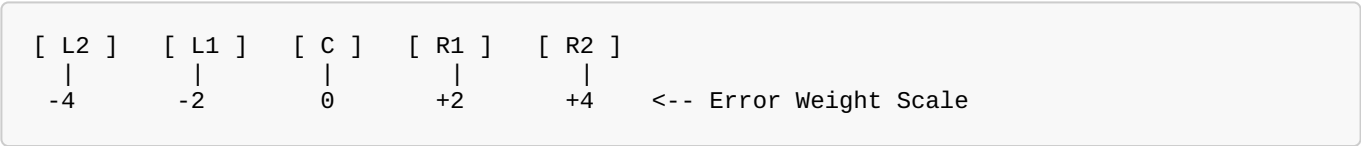
B. L298N Motor Driver to Arduino

L298N Pin	Arduino Pin	Function
ENA	D9 (PWM)	Left Motor Speed Control
IN1	D8	Left Motor Forward
IN2	D7	Left Motor Backward
IN3	D6	Right Motor Forward
IN4	D5	Right Motor Backward
ENB	D10 (PWM)	Right Motor Speed Control

Power Logic: Connect the 12V terminal of the L298N to the battery positive. Connect L298N GND to battery negative AND Arduino GND. Connect L298N 5V output to Arduino VIN (if powering Arduino via the driver).

4. Sensor Logic & Weighted Error Algorithm

Unlike basic 2-IR followers that only execute hard left/right turns, a 5-IR array provides gradient error values. The logic assigns numerical weights to specific sensor activations:



When the robot deviates from the black line, the error value shifts positively or negatively. If the line is completely lost, the algorithm recalls the `lastError` state to perform a sharp pivot, searching for the track.

$$MotorSpeed = BaseSpeed \pm (Kp \times Error + Kd \times \Delta Error)$$

5. Complete Arduino Source Code (C++)

Below is the optimized PID-based control script for the 5-IR setup.

```

/**
 * 5-Channel IR Line Follower Robot (PID Control)
 * Hardware: Arduino Uno/Nano, 5-IR Array, L298N Motor Driver
 */

// Motor Driver Pins (L298N)
#define ENA 9 // Left Motor Speed
#define IN1 8 // Left Motor Forward
#define IN2 7 // Left Motor Backward
#define IN3 6 // Right Motor Forward
#define IN4 5 // Right Motor Backward
#define ENB 10 // Right Motor Speed

// 5 IR Sensor Array Pins (Left to Right)
#define IR_L2 2 // Far Left
#define IR_L1 3 // Near Left
#define IR_C 4 // Center
#define IR_R1 11 // Near Right
#define IR_R2 12 // Far Right

// Base Speed and PID Constants
int baseSpeed = 120;
int maxSpeed = 200;

float Kp = 25.0;
float Kd = 15.0;
float Ki = 0.0;

int lastError = 0;
int integral = 0;

void setup() {
    Serial.begin(9600);

    // Motor Output Pins
    pinMode(ENA, OUTPUT);
    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);
    pinMode(IN3, OUTPUT);
    pinMode(IN4, OUTPUT);
    pinMode(ENB, OUTPUT);

    // Sensor Input Pins
    pinMode(IR_L2, INPUT);
    pinMode(IR_L1, INPUT);
    pinMode(IR_C, INPUT);
    pinMode(IR_R1, INPUT);
    pinMode(IR_R2, INPUT);
}

void loop() {
    int error = calculateError();

    int P = error;
    int I = integral + error;
    int D = error - lastError;

    int PID_value = (Kp * P) + (Ki * I) + (Kd * D);

    lastError = error;
    integral = I;

    int leftMotorSpeed = baseSpeed + PID_value;
    int rightMotorSpeed = baseSpeed - PID_value;

    // Constrain speeds
    leftMotorSpeed = constrain(leftMotorSpeed, 0, maxSpeed);
    rightMotorSpeed = constrain(rightMotorSpeed, 0, maxSpeed);

    setMotorSpeeds(leftMotorSpeed, rightMotorSpeed);
}

// Calculate position error based on 5 IR sensors

```

```

// Assuming logic LOW (0) means black line, HIGH (1) means white surface
int calculateError() {
    int l2 = !digitalRead(IR_L2);
    int l1 = !digitalRead(IR_L1);
    int c  = !digitalRead(IR_C);
    int r1 = !digitalRead(IR_R1);
    int r2 = !digitalRead(IR_R2);

    int error = 0;

    // Weighted error calculation
    if (l2 == 1 && l1 == 0 && c == 0 && r1 == 0 && r2 == 0) error = -4;    // Sharp Left
    else if (l2 == 1 && l1 == 1 && c == 0 && r1 == 0 && r2 == 0) error = -3;
    else if (l2 == 0 && l1 == 1 && c == 0 && r1 == 0 && r2 == 0) error = -2; // Slight Left
    else if (l2 == 0 && l1 == 1 && c == 1 && r1 == 0 && r2 == 0) error = -1;
    else if (l2 == 0 && l1 == 0 && c == 1 && r1 == 0 && r2 == 0) error = 0;  // Center
    else if (l2 == 0 && l1 == 0 && c == 1 && r1 == 1 && r2 == 0) error = 1;
    else if (l2 == 0 && l1 == 0 && c == 0 && r1 == 1 && r2 == 0) error = 2;  // Slight Right
    else if (l2 == 0 && l1 == 0 && c == 0 && r1 == 1 && r2 == 1) error = 3;
    else if (l2 == 0 && l1 == 0 && c == 0 && r1 == 0 && r2 == 1) error = 4;  // Sharp Right

    // If line is lost (all white), maintain previous turn direction strongly
    else if (l2 == 0 && l1 == 0 && c == 0 && r1 == 0 && r2 == 0) {
        if (lastError < 0) error = -5;
        else if (lastError > 0) error = 5;
    }

    return error;
}

void setMotorSpeeds(int leftSpeed, int rightSpeed) {
    // Left Motor
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    analogWrite(ENA, leftSpeed);

    // Right Motor
    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);
    analogWrite(ENB, rightSpeed);
}

```